설비 고장을 예측하는 AI 데이터 분석

함수 — 매개변수와 반환값

매개변수의 이해 → 여러 인자와 키워드 인자 → 반환값의 이해

posco

CONTENTS

목차

01 매개변수의 이해

02 여러 인자와 키워드 인자

03 반환값의 이해

posco

01

매개변수의 이해

함수가 외부로부터 값을 받는 통로

고정된 값만 쓰는 함수의 한계

함수 안에 값을 고정하면 그 값 하나에만 쓰는 반쪽 함수다.

한계 01

함수 안에 값을 고정하면 그 값 하나에만 쓰는 반쪽 함수

한계 02

설비마다 함수를 새로 만들면 없앤 중복이 다시 생긴다

압축기A용 함수에 압축기A가 고정돼 있으면, 펌프1용 함수를 또 만들어야 한다.

함수가 하나의 값에 묶이면

함수가 하나의 값에만 묶이면 재사용의 의미가 절반으로 줄어든다.

01 · 재사용 절반

같은 작업을 다른 값에 적용할 수 없음

재사용의 의미가 절반으로 줄어든다.

02 · 중복 재발

설비마다 함수를 복사하면 또 중복 발생

함수로 없앤 중복이 값 고정 때문에 되살아난다.

03 · 우리의 목표

함수 하나로 모든 설비를 처리하기

설비 이름을 함수 밖에서 그때그때 넣어 줄 수 있다면 가능하다.

매개변수가 필요한 이유

매개변수는 함수가 외부로부터 값을 받는 통로다.

POINT 01

매개변수는 함수가 외부로부터 값을 받는 통로

POINT 02

같은 함수가 받는 값에 따라 다른 일을 한다

자판기에 동전을 넣으면 그 동전이 안으로 들어가 작동하듯, 호출 시 값이 매개변수를 통해 들어온다.

같은 처리를 다른 값에

매개변수가 있으면 설비 이름을 받는 함수 하나면 충분하다.

매개변수가 있으면 **설비 이름을 받는 함수 하나**면 충분하다.

압축기A를 넣어 부르면 **압축기A를**, 펌프1을 넣으면 **펌프1을** 처리한다.

믹서기에 사과를 넣으면 사과주스, 딸기를 넣으면 딸기주스가 나오는 것과 같다.

> **다른 값에 같은 처리를 적용하는 재사용 — 이것이 함수의 진정한 재사용이다.**

매개변수와 인자의 차이

매개변수는 함수를 만들 때 적는 받을 변수, 인자는 부를 때 넣는 실제 값이다.

매개변수 · PARAMETER

함수 정의 시 괄호 안에 적는 받을 변수

```
def check(name):
```

받을 준비가 된 빈 자리, 아직 구체적인 값 없음

인자 · ARGUMENT

함수 호출 시 괄호 안에 넣는 실제 값

```
check("압축기A")
```

매개변수라는 빈 자리에 채워지는 구체적인 값

등장 시점이 다르다

매개변수는 정의할 때, 인자는 호출할 때 등장한다.

구분	매개변수	인자
시점	정의할 때	호출할 때
위치	def 괄호	호출 괄호
정체	받을 변수	실제 값

> 매개변수는 택배 받을 주소를 적어 둔 빈 상자, 인자는 그 상자에 담겨 오는 물건.

매개변수 정의와 전달 구조

괄호 안에 받을 값의 이름을 적으면 함수가 그 값을 받을 준비를 한다.

CODE

```
def check(name):
    print(name + " 점검을 시작합니다")
check("압축기A")
```

정의와 호출은 짝을 이룬다

정의할 때 매개변수가 하나면, 호출할 때 인자도 하나를 넣어야 한다.

01 · 정의의 매개변수

받을 자리, def 괄호 안 이름

함수를 만들 때 빈 자리로 준비해 둔다.

02 · 호출의 인자

그 자리에 들어갈 실제 값

함수를 부를 때 빈 자리에 채워 넣는다.

03 · 개수 일치

매개변수 한 개면 인자도 한 개

안 넣으면 missing argument, 더 넣으면 too many arguments 에러.

빈 괄호에 이름 하나로 함수가 달라진다

빈 괄호에 이름 하나 적었을 뿐인데 함수가 외부와 소통하기 시작한다.

KEY 01

빈 괄호에 이름 하나 적었을 뿐인데 함수가 외부와 소통

KEY 02

같은 함수가 다양한 값을 받아 처리할 수 있게 됨

posco

인자가 전달되는 흐름

호출 → 준비 → 전달 → 실행 → 종료의 다섯 단계로 흐른다.

STEP 1

호출

인자를 넣어 함수를 호출한다.

check("압축기A")

→

STEP 2

준비

인자 값이 만들어진다.

"압축기A" 준비

→

STEP 3

전달

매개변수 자리에 값이 채워진다.

name = "압축기A"

→

STEP 4

실행

채워진 값으로 함수 코드가 실행된다.

print(name + ...)

→

STEP 5

종료

함수가 끝나면 매개변수도 사라진다.

name 사라짐

name = 압축기A라는 대입이 함수 안에서 자동으로 일어나는 것과 같다.

매개변수는 매번 새로 채워진다

인자가 매개변수에 복사되어 함수 안에서 그 값을 쓸 수 있게 된다.

인자가 매개변수에 **복사**되어 함수 안에서 그 값을 쓸 수 있게 된다.

호출할 때마다 매개변수는 새 인자로 **다시 채워지고** 이전 값은 남지 않는다.

함수는 매번 **깨끗한 상태**에서 새 인자를 받아 동작한다.

> 함수 안에서 매개변수를 바꿔도 함수 밖의 원래 값은 그대로 남는다.

매개변수 이름 정하기

매개변수 이름은 받을 값의 의미를 드러내야 한다.

01 · 의미 담기

받을 값의 의미를 드러내는 이름

그 자리에 어떤 값이 들어올지 이름만 봐도 알 수 있게.

02 · 예

설비명: `name`

온도: `temperature`

의미를 드러내는 영어 단어를 그대로 쓴다.

03 · 단순 계산

두 수 더하기는 a, b처럼 짧아도 무방

값의 의미가 단순할 땐 짧은 이름이 더 읽기 좋다.

좋은 이름과 나쁜 이름

매개변수 이름은 함수를 쓰는 사람에게 일종의 안내문 역할을 한다.

용도	권장	비권장
설비명	`name`	`a`
온도값	`temperature`	`x`
진동값	`vibration`	`v1`

> def check(equipment_name) — 이름만 봐도 설비 이름을 넣어야 한다는 걸 알 수 있다.

실습 1. 매개변수 1개 함수 만들기

매개변수 하나를 가진 함수를 만들고 여러 인자로 호출하기

단계

① def 괄호 안에 매개변수 이름 하나를 적어 함수 정의
② 함수 안에서 매개변수를 print에 활용
③ 서로 다른 인자로 여러 번 호출
④ 인자에 따라 결과가 달라지는지 확인

예상 결과

압축기A 점검 시작
펌프1 점검 시작

posco

실습 1. 매개변수 1개 함수 만들기 정답

CODE · 정답

```
def check(name):
    print(name + " 점검 시작")
check("압축기A")
check("펌프1")
```

02

여러 인자와 키워드 인자

인자 순서가 중요한 이유와 키워드로 자리를 정하는 법

매개변수 여러 개 쓰기

괄호 안에 매개변수 두 개를 쉼표로 나열한다.

CODE

```
def report(name, temp):
    print(name + "의 온도: " + str(temp) + "도")
report("압축기A", 75.3)
```

매개변수 개수에 제한은 없다

매개변수 개수에는 정해진 제한이 없다. 다만 순서를 챙겨야 한다.

RULE 01

두 개든 세 개든 필요한 만큼 쉼표로 나열 가능

RULE 02

매개변수 두 개면 인자도 두 개를 넣어야 함

너무 많아지면 인자를 순서대로 챙기기 어려우니 적절히 유지하는 게 좋다.

인자 순서가 중요한 이유

어떤 인자가 어떤 매개변수로 들어갈지는 순서로 결정된다.

호출	name	temp
`report("압축기A", 75.3)`	`"압축기A"`	`75.3`
`report(75.3, "압축기A")`	`75.3`	`"압축기A"`

위치 인자(positional argument) — 위치에 따라 전달되는 인자.
숫자끼리는 순서를 틀려도 에러 없이 조용히 틀린 결과가 나온다.

위치 인자는 순서가 생명

위치 인자는 편리하지만 순서에 의존한다.

KEY 01

위치 인자는 적은 순서대로 매개변수에 차례로 들어감

KEY 02

매개변수가 많아지면 순서를 헷갈리기 쉬움

이 문제를 덜어 주는 방법이 뒤에 배울 키워드 인자다.

매개변수 개수와 인자 개수

매개변수 수만큼 인자를 넣어야 한다는 기본 규칙.

매개변수	인자	결과
2개	2개	정상 동작
2개	1개	인자 부족 에러
2개	3개	인자 초과 에러

> 받을 자리는 있는데 줄 값이 없으니 일을 시작할 수 없고, 남는 값은 어디에 둘지 모르니 멈춘다.

개수 일치는 안전장치

개수 일치 규칙은 번거롭게 느껴질 수 있지만 사실 안전장치다.

SAFETY 01

값을 빠뜨리거나 잘못 넣으면 즉시 에러로 알려줌

SAFETY 02

잘못된 결과가 조용히 나오는 것보다 멈춰서 알려주는 게 안전

기본값 인자로 일부 인자를 생략할 수 있는 방법은 뒤에서에서 다룬다.

실습 2. 다중 매개변수로 센서값 계산하기

매개변수 두세 개를 가진 함수로 센서 데이터를 처리하기

단계

① def 괄호 안에 매개변수 두 개를 쉼표로 정의
② 함수 안에서 두 매개변수를 함께 활용
③ 인자 두 개를 순서대로 전달해 호출
④ 인자 순서를 바꾸면 결과가 어떻게 달라지는지 확인

예상 결과

모터 78 도
펌프 92 도

posco

실습 2. 다중 매개변수로 센서값 계산하기 정답

CODE · 정답

```
def report(name, temp):
    print(name, temp, "도")
report("모터", 78)
report("펌프", 92)
```

키워드 인자

위치가 아니라 이름으로 값의 자리를 정한다.

CODE

```
def report(name, temp):
    print(name + ": " + str(temp))
report(name="압축기A", temp=75.3)
report(temp=75.3, name="압축기A")
```

키워드 인자의 두 장점

키워드 인자는 호출하는 줄만 봐도 무슨 값인지 드러난다.

MERIT 01

이름으로 자리를 지정하니 순서에 자유로움

MERIT 02

호출하는 줄만 봐도 무슨 값인지 바로 드러남

report(name="압축기A", temp=75.3) — 75.3이 온도라는 게 호출하는 줄에서 바로 보인다.

위치 인자와 키워드 인자 비교

위치 인자는 간결, 키워드 인자는 명확. 상황에 맞게 고른다.

구분	위치 인자	키워드 인자
전달 기준	순서	이름
표기	`add(3, 4)`	`add(a=3, b=4)`
장점	간결함	명확함
적합	인자 적을 때	인자 많을 때

> 섞어 쓸 때는 위치 인자가 먼저. `add(3, b=4)`는 OK, `add(a=3, 4)`는 에러.

섞어 쓸 때는 위치 먼저

위치 인자가 키워드 인자보다 앞에 와야 Python이 헷갈리지 않는다.

RULE 01

섞어 쓸 때 위치 인자가 키워드 인자보다 앞에 와야 함

RULE 02

입문 단계에서는 한 방식으로 통일해 쓰는 게 안전

실습 3. 키워드 인자로 함수 호출하기

키워드 인자로 이름을 지정해 순서와 무관하게 호출하기

단계

① 매개변수 두 개를 가진 함수를 정의
② 호출할 때 매개변수 이름을 지정해 값을 전달
③ 키워드로 전달하면 순서를 바꿔도 같은 결과인지 확인
④ 위치 인자와 키워드 인자를 섞을 때는 위치가 먼저임을 확인

예상 결과

모터 78 (순서 무관)
펌프 92

posco

실습 3. 키워드 인자로 함수 호출하기 정답

CODE · 정답

```
def report(name, temp):
    print(name, temp)
report(temp=78, name="모터")   # 키워드 → 순서 무관
report("펌프", temp=92)         # 위치 먼저, 키워드 나중
```

03

반환값의 이해

print와 return의 차이부터 여러 값 반환까지

함수 결과를 다시 사용하려면

함수가 계산한 결과를 이어 쓰려면 화면 출력만으로는 부족하다.

함수가 계산한 결과를 **이어 쓰려면 화면 출력만으로는 부족**하다.

평균을 화면에 출력만 하면 **그 값을 다른 계산에 쓸 수 없다**.

결과를 프로그램이 받아 변수에 저장하거나 다음 계산에 넘기려면 **돌려받아야** 한다.

> 함수에 값을 넣는 것이 입력이라면, 함수가 값을 돌려주는 것이 출력이다.

함수가 결과를 돌려줘야 하는 이유

결과를 돌려줘야 그 결과를 가지고 다음 일을 할 수 있다.

REASON 01

자판기가 음료를 보여주기만 하면 마실 수 없음

REASON 02

함수도 결과를 돌려줘야 다음 일을 할 수 있음

print와 return의 결정적 차이

print는 화면에 보여주고, return은 값을 돌려준다.

구분	print	return
하는 일	화면에 보여줌	값을 돌려줌
대상	사람 눈	프로그램
이후 사용	불가능	변수 저장
함수 실행	계속됨	즉시 종료

return은 값이 프로그램 안에서 살아 있게 한다.

print는 사람에게, return은 프로그램에게

print는 사람에게 보여주는 것, return은 프로그램에게 돌려주는 것이다.

SUMMARY 01

print는 사람에게, return은 프로그램에게

SUMMARY 02

결과를 이어 써야 한다면 반드시 return이 필요

함수 안에서 return으로 돌려주고 부른 쪽에서 print로 출력할 수도 있다.

return 구조와 값 받기

함수 안에서 돌려주고 싶은 값 앞에 return을 적는다.

CODE

```
def add(a, b):
    return a + b
total = add(3, 4)
print(total)
```

값을 받는 두 가지 방법

함수 호출 자체가 돌려진 값으로 취급된다.

01 · 변수에 담기

total = add(3, 4)

나중에 여러 번 사용

```
total = add(3, 4)
print(total)
print(total * 2)
```

02 · 바로 쓰기

print(add(10, 20))

곧장 다른 곳에 전달

```
print(add(10, 20))
# add가 돌려준 값이
# 곧장 print로 전달
```

돌려주고 받는 것은 짝

돌려주는 쪽과 받는 쪽이 짝을 이뤄야 의미가 있다.

PAIR 01

return으로 돌려줘도 받지 않으면 값은 사라짐

PAIR 02

돌려주는 쪽과 받는 쪽이 짝을 이뤄야 의미가 있음

total = add(3, 4) 없이 그냥 add(3, 4)만 쓰면 돌려진 값이 아무 데도 담기지 않고 사라진다.

반환값을 변수에 담기

변수에 담는 순간 함수의 결과가 프로그램 안에서 살아 움직인다.

CODE

```
def calc_average(a, b):
    return (a + b) / 2
avg = calc_average(75.3, 88.0)
print("평균 온도:", avg)
```

담은 값은 일반 변수처럼

함수가 돌려준 값을 변수에 담으면 결과를 이어서 쓸 수 있다.

함수가 돌려준 값을 변수에 담으면 **결과를 이어서 쓸 수 있다**.

평균을 담아 두고 **기준 초과 판정**에 쓰고 보고서 출력에도 사용한다.

변수 이름은 **avg**처럼 그 값이 무엇인지 드러나게 짓는다.

> 함수가 돌려준 하나의 값을 여러 곳에서 반복해 쓴다.

posco

return의 함수 종료 효과

return을 만나면 값을 돌려주고 바로 함수를 빠져나온다.

CODE

```
def check(temp):
    return temp
    print("실행 안 됨")   # ← 이 줄은 영원히 실행되지 않음
print(check(75))
```

return은 두 가지를 동시에

return은 값을 돌려주는 동시에 함수를 즉시 끝낸다.

EFFECT 01

return은 값을 돌려주는 동시에 함수를 즉시 끝냄

EFFECT 02

실행되어야 할 코드는 모두 return보다 위에 두기

조건문과 함께 쓰면 특정 조건에서 함수를 일찍 끝내는 데 유용하다.

실습 4. 반환값으로 간단 계산기 만들기

print가 아니라 return으로 결과를 돌려주고 변수에 담기

단계

① 값을 받아 계산하는 함수를 정의
② 계산 결과를 print가 아니라 return으로 돌려주기
③ 호출 결과를 변수에 담기
④ 담은 값을 다음 계산·출력에 이어 쓰기

예상 결과

85.0
90.0 (담은 값을 이어 씀)

실습 4. 반환값으로 간단 계산기 만들기 정답

CODE · 정답

```
def calc_avg(a, b):
    return (a + b) / 2
avg = calc_avg(80, 90)
print(avg)       # 85.0
print(avg + 5)   # 90.0 (담은 값을 이어 씀)
```

여러 값을 한 번에 반환하기

return 뒤에 쉼표로 나열하면 여러 값이 튜플로 묶여 돌아온다.

CODE

```
def min_max(values):
    return min(values), max(values)
result = min_max([75.3, 88.0, 49.1])
print(result)
```

튜플로 묶여 돌아온다

호출 한 번으로 여러 정보를 얻을 수 있어 데이터 분석에 유용하다.

POINT 01

return 뒤에 쉼표로 나열하면 여러 값이 튜플로 묶임

POINT 02

호출 한 번으로 여러 정보를 얻을 수 있어 분석에 유용

센서 데이터 통계 — 최솟값·최댓값·평균을 한 함수로 한꺼번에 구해 돌려준다.

반환값 언패킹으로 받기

받는 쪽에서 변수를 쉼표로 나열하면 각 값이 순서대로 따로 담긴다.

CODE

```
def min_max(values):
    return min(values), max(values)
low, high = min_max([75.3, 88.0, 49.1])
```

posco

변수 개수와 값 개수를 맞추기

묶어서 돌려주고 풀어서 받는 깔끔한 방식이다.

RULE 01

언패킹할 때 변수 개수와 돌아오는 값 개수가 같아야 함

RULE 02

묶어서 돌려주고 풀어서 받는 깔끔한 방식

두 값이 돌아오는데 변수를 세 개 적거나 한 개만 적으면 에러가 난다.

반환값이 없는 함수와 None

return이 없는 함수도 호출하면 None을 돌려준다.

CODE

```
def greet(name):
    print(name + "님 환영합니다")
result = greet("작업자")
print(result)   # ← None 출력
```

posco

None이 알려주는 흔한 실수

None을 알아 두면 흔한 실수를 피할 수 있다.

MISTAKE 01

return 없이 print만 한 함수의 결과를 담으면 None이 담김

LESSON 02

결과를 이어 쓰려면 반드시 return이 필요

화면에는 글자가 보였지만 변수에는 None이 들어간 것이다.

실습 5. 센서 통계 함수 만들기

목록을 받아 여러 통계를 한 번에 돌려주고 언패킹으로 나눠 받기

단계

① 센서값 목록을 매개변수로 받는 함수를 정의
② min·max·합÷개수로 최소·최대·평균을 계산
③ 세 값을 쉼표로 함께 return
④ 돌려받은 값을 세 변수로 언패킹해 출력

예상 결과

78 92 85.0

실습 5. 센서 통계 함수 만들기 정답

CODE · 정답

```
def stats(v):
    return min(v), max(v), sum(v) / len(v)
lo, hi, avg = stats([78, 92, 85])
print(lo, hi, avg)   # 78 92 85.0
```